

CPT102-2526 Exercise 9 - soln

Depth-First and Breadth-First Search

In Lecture 9, we have solved several graph problems, such as **s-t connectivity**, **single-source all paths**, and **shortest path in undirected graphs**. It turns out that DFS and BFS can also be modified to solve other graph problems!

You may write your algorithm in either pseudocode or Java. Then, justify its correctness and explain how it satisfies the required worst-case running time.

Problem 9.1 Cycle Detection

A graph is **acyclic** if it contains no cycle, where a cycle is a path that starts and ends at the same vertex without repeating edges. Given an undirected graph, determine whether it is acyclic; if the graph contains a cycle, return the vertices of one such cycle. The required worst-case running time is $O(V + E)$.

Hint: During DFS, think carefully about what it means to revisit a vertex that is not the one you just came from.

Solution:

Run DFS in every component. If DFS sees an edge from the current vertex to a vertex that has already been visited and that vertex is not the parent we just came from, the edge closes a cycle. Parent links reconstruct the cycle.

FIND-CYCLE(G):

```
marked[v] <- false for every vertex v
edgeTo[v] <- undefined for every vertex v
cycle <- empty
seenNeighbor[x] <- false for every vertex x
```

```
For each vertex v in G:
```

```
  For each vertex w in Adj(v):
```

```
    If seenNeighbor[w] is true:
```

```
      Return "not acyclic" and the cycle [v, w, v]
```

```
    seenNeighbor[w] <- true
```

```
  Reset seenNeighbor[w] to false for every w in Adj(v)
```

```
marked[v] <- false for every vertex v
```

```
For each vertex v in G:
```

```
  If marked[v] is false:
```

```
    DFS-CYCLE(parent = NONE, v)
```

```
    If cycle is not empty:
```

```

        Return "not acyclic" and cycle

    Return "acyclic"

DFS-CYCLE(parent, v):
    marked[v] <- true
    For each vertex w in Adj(v):
        If cycle is not empty: return

        If marked[w] is false:
            edgeTo[w] <- v
            DFS-CYCLE(parent = v, w)

        Else if w is not parent:
            cycle <- empty list
            x <- v
            While x is not w:
                Add x to cycle
                x <- edgeTo[x]
            Add w to cycle
            Add v to cycle      # closes the cycle
    return

```

During DFS, `edgeTo` records the DFS-tree edge by which each vertex was first discovered. If a vertex v finds an already marked neighbor w that is not v 's parent, then the DFS-tree path from v back to w together with the edge v - w forms a closed path, so the reconstructed list is a valid cycle. Conversely, in an undirected DFS, any cycle must eventually contain an edge leading to a previously visited vertex other than the immediate parent, so the algorithm detects a cycle whenever one exists. If no such event occurs in any component, the DFS forest has no non-tree cycle-forming edge, so the graph is acyclic.

DFS marks every vertex once and examines every adjacency-list entry a constant number of times, so it takes $O(V + E)$. The total worst-case running time is $O(V + E)$.

Problem 9.2 Bipartite Graph / Two-colorability

A graph is **bipartite** if its vertices can be divided into two sets such that every edge has one endpoint in each set. Given an undirected graph, determine whether it is bipartite. If it is, return a valid bipartition; otherwise, return an odd-length cycle proving that no such bipartition exists. The required worst-case running time is $O(V + E)$.

Note: This also implies that the graph is 2-colorable: the vertices can be assigned one of two colors in such a way that no edge connects vertices of the same color.

Hint: Try assigning opposite labels across every edge as you explore, and watch for an edge whose endpoints already have the same label.

Solution:

Run DFS in every component while assigning each newly discovered vertex the opposite color of the vertex that discovered it. If an edge ever connects two vertices with the same color, that edge plus the DFS-tree path between them is an odd-length cycle. Otherwise, the two color classes form a valid bipartition.

FIND-BIPARTITION-OR-ODD-CYCLE(G):

```
marked[v] <- false for every vertex v
color[v] <- false for every vertex v
edgeTo[v] <- undefined for every vertex v
oddCycle <- empty
```

```
For each vertex v in G:
```

```
  If marked[v] is false:
```

```
    color[v] <- false
```

```
    DFS-BIPARTITE(v)
```

```
    If oddCycle is not empty:
```

```
      Return "not bipartite" and oddCycle
```

```
leftSet <- all vertices v with color[v] = false
```

```
rightSet <- all vertices v with color[v] = true
```

```
Return "bipartite" and the bipartition (leftSet, rightSet)
```

DFS-BIPARTITE(v):

```
marked[v] <- true
```

```
For each vertex w in Adj(v):
```

```
  If oddCycle is not empty: return
```

```
  If marked[w] is false:
```

```
    edgeTo[w] <- v
```

```
    color[w] <- opposite of color[v]
```

```
    DFS-BIPARTITE(w)
```

```
  Else if color[w] = color[v]:
```

```
    oddCycle <- empty list
```

```
    Add w to oddCycle
```

```
    x <- v
```

```
    While x is not w:
```

```
      Add x to oddCycle
```

```
      x <- edgeTo[x]
```

```
    Add w to oddCycle # closes the cycle
```

```
return
```

The DFS maintains the invariant that every tree edge connects vertices of opposite colors. Therefore, if DFS finishes without finding a same-color edge, then every edge in the graph has endpoints in different color classes, so the two classes are a valid bipartition. If DFS finds an edge $v-w$ whose endpoints already have the same color, then the DFS-tree path from v to w has even length because colors alternate along the tree path and both endpoints have the same color. Adding the edge $v-w$ creates a cycle with odd length, so the returned cycle proves that no bipartition exists. A graph is bipartite exactly when it has no odd-length cycle.

DFS marks each vertex once and examines each adjacency-list entry a constant number of times. Building the final two sets and reconstructing one cycle are linear in the number of vertices. The total worst-case running time is $O(V + E)$.

Problem 9.3 Connected Components

A **connected component** is a maximal set of vertices such that every pair of vertices in the set is connected by some path. Given an undirected graph, compute all of its connected components: report how many components there are, identify which component each vertex belongs to, give the size of each component, and answer whether two vertices are connected. The required worst-case running time is $O(V + E)$.

Hint: You can always start a new search from multiple vertices, and you can make use of component id.

Solution:

Start a DFS from every unmarked vertex. Each such start begins one new connected component. During that DFS, assign the same component id to every reached vertex and count how many vertices receive that id.

```
COMPUTE-CONNECTED-COMPONENTS(G):  
    marked[v] <- false for every vertex v  
    id[v] <- undefined for every vertex v  
    size[c] <- 0 for every possible component id c  
    count <- 0
```

```
    For each vertex v in G:  
        If marked[v] is false:  
            DFS-COMPONENT(v, count)  
            count <- count + 1  
    Return count, id, and size
```

```

DFS-COMPONENT(v, componentId):
    marked[v] <- true
    id[v] <- componentId
    size[componentId] <- size[componentId] + 1
    For each vertex w in Adj(v):
        If marked[w] is false:
            DFS-COMPONENT(w, componentId)

```

A DFS starting at an unmarked vertex visits exactly the vertices reachable from that start vertex. Thus all vertices assigned the current component id are mutually connected. When the search ends, no unmarked vertex is reachable from that start vertex; otherwise DFS would have visited it. Therefore the set assigned that id is maximal and is exactly one connected component. Repeating this process from every remaining unmarked vertex finds every component once. Two vertices are connected exactly when they were reached in the same DFS, so `CONNECTED(v, w)` is correct.

Each vertex is marked once, and every edge appears in two adjacency lists and is examined a constant number of times. Computing all components takes $O(V + E)$.

Problem 9.4 Biconnectivity

An **articulation vertex**, or **cut vertex**, is a vertex whose removal increases the number of connected components. A graph is **biconnected** if it has no articulation vertices. Given an undirected graph, find all articulation vertices and determine whether the graph is biconnected. The required worst-case running time is $O(V + E)$.

Hint: During DFS, think about what happened if we compare when a vertex is first discovered with the earliest-discovered vertex reachable from its descendants.

Solution:

Run DFS and give every vertex a discovery number `pre[v]`. Also compute `low[v]`, the smallest discovery number reachable from `v` or from any descendant of `v` using zero or more DFS-tree edges followed by at most one back edge. A non-root vertex `v` is an articulation vertex if some child subtree cannot reach a proper ancestor of `v`. A DFS root is an articulation vertex if it has more than one DFS child.

```

FIND-ARTICULATION-VERTICES(G):
    pre[v] <- -1 for every vertex v
    low[v] <- -1 for every vertex v
    articulation[v] <- false for every vertex v
    time <- 0

    For each vertex v in G:
        If pre[v] = -1:
            DFS-ARTICULATION(parent = v, v)

```

```

articulationSet <- all vertices v with articulation[v] = true
isBiconnected <- true exactly when articulationSet is empty
Return articulationSet and isBiconnected

```

```

DFS-ARTICULATION(parent, v):
  children <- 0
  pre[v] <- time
  low[v] <- pre[v]
  time <- time + 1
  For each vertex w in Adj(v):
    If pre[w] = -1:
      children <- children + 1
      DFS-ARTICULATION(parent = v, w)

      low[v] <- minimum(low[v], low[w])

      If parent is not v and low[w] >= pre[v]:
        articulation[v] <- true

    Else if w is not parent:
      low[v] <- minimum(low[v], pre[w])

  If parent is v and children > 1:
    articulation[v] <- true

```

The value $low[v]$ is correct by induction on the DFS recursion: it starts as $pre[v]$, is lowered by back edges from v to already discovered non-parent vertices, and is also lowered by the low values of child subtrees. For a non-root vertex v , if a child w has $low[w] \geq pre[v]$, then the subtree rooted at w has no back edge to an ancestor of v . Removing v separates that child subtree from the rest of the graph, so v is an articulation vertex. If every child w has $low[w] < pre[v]$, each child subtree has a route back above v , so removing v does not separate any child subtree from the rest. For a DFS root, there is no ancestor to connect its different DFS child subtrees; therefore the root is an articulation vertex exactly when it has more than one DFS child. Thus the algorithm finds exactly all articulation vertices and correctly reports the graph as biconnected, according to the exercise definition, when no articulation vertex exists.

DFS assigns pre and low once per vertex and examines each adjacency-list entry a constant number of times. The total worst-case running time is $O(V + E)$.

Problem 9.5 Diameter of a Tree

The **diameter** of a tree is the length of the longest path between any two vertices, where length is measured by number of edges. Given an undirected tree (connected

and acyclic graph), find the two endpoints of a diameter and report its length. The required worst-case running time is $O(V + E)$.

Hint: You could begin by picking an arbitrary node and find its farthest node.

Solution:

Pick any vertex v . Compute the shortest path from v to every other vertex using BFS. Let w be the vertex with the largest shortest path distance and w must be one end of the diameter. Compute the shortest path from w to every other vertex using BFS. Let x be the vertex with the largest shortest path distance. The path from w to x gives the diameter of the tree, since x is the other end of the diameter. Since we use 2 BFS calls, the running time is $O(V + E)$.

This is the end of CPT102-2526 Exercise 9 **Soln**.